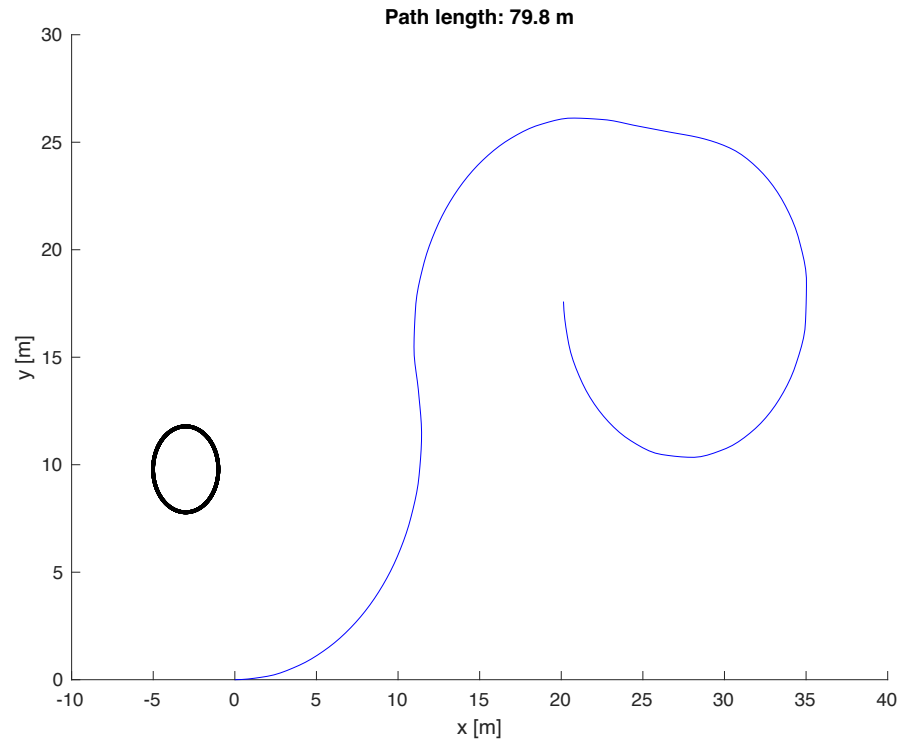


Model Predictive Control for Autonomous Vehicles

TSFS12: Autonomous Vehicles – Planning,
Control, and Learning Systems

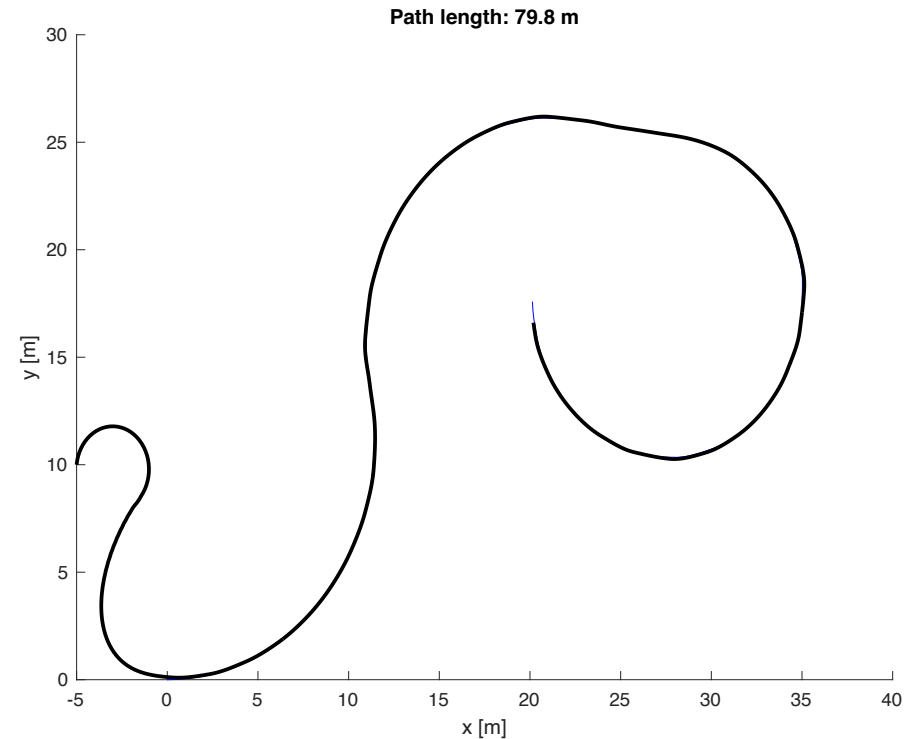
Lecture 7

From Lecture 6: Linear and non-linear controllers



Linear controller

$$u = u_0 - k_1 d - k_2 \theta_e$$



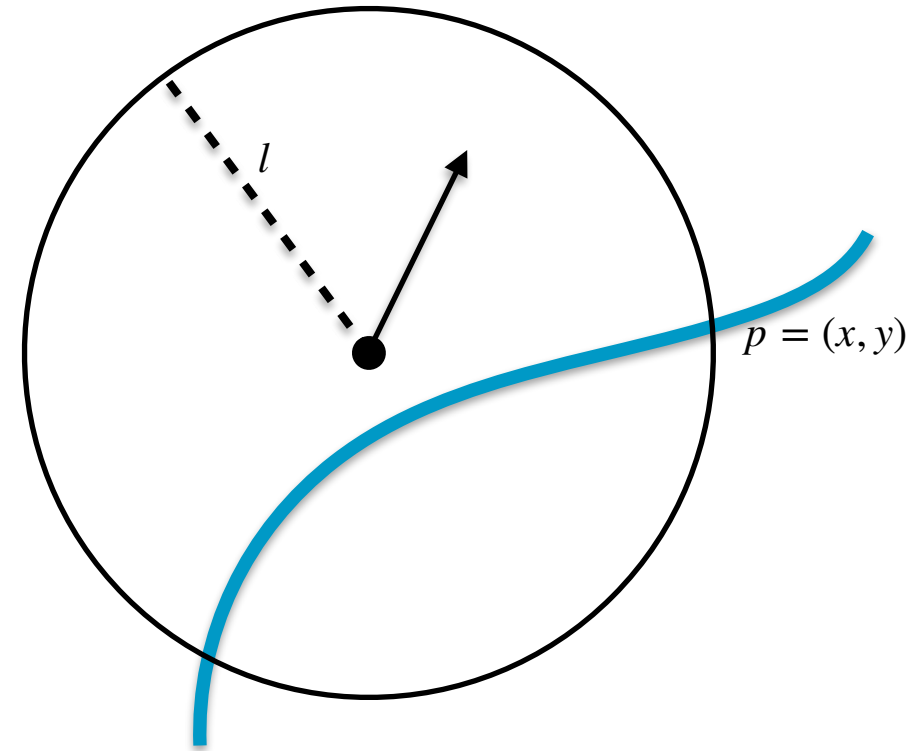
Non-linear controller

$$u = u_0 - k_1 \frac{\sin \theta_e}{\theta_e} d - k_2 \theta_e$$

From Lecture 6: Pure-pursuit, summary

1. Determine intersection point p between path and look-ahead circle (with radius l)
2. Determine coordinates of p in vehicle-local coordinate system $p = (x, y)$
3. Steering angle δ for single-track model with wheel-base L

$$\tan \delta = -L \frac{2x}{l^2}$$



Linear Quadratic Regulators

- Cost function from last slide

$$J = \sum_{t=0}^{\infty} x_t^T Q x_t + u_t^T R u_t,$$

- Matrices Q and R are the tuning variables for the controller
- For the path follower $x = (d, \theta_e)$ and $u = \text{curvature} = \kappa_t$. With

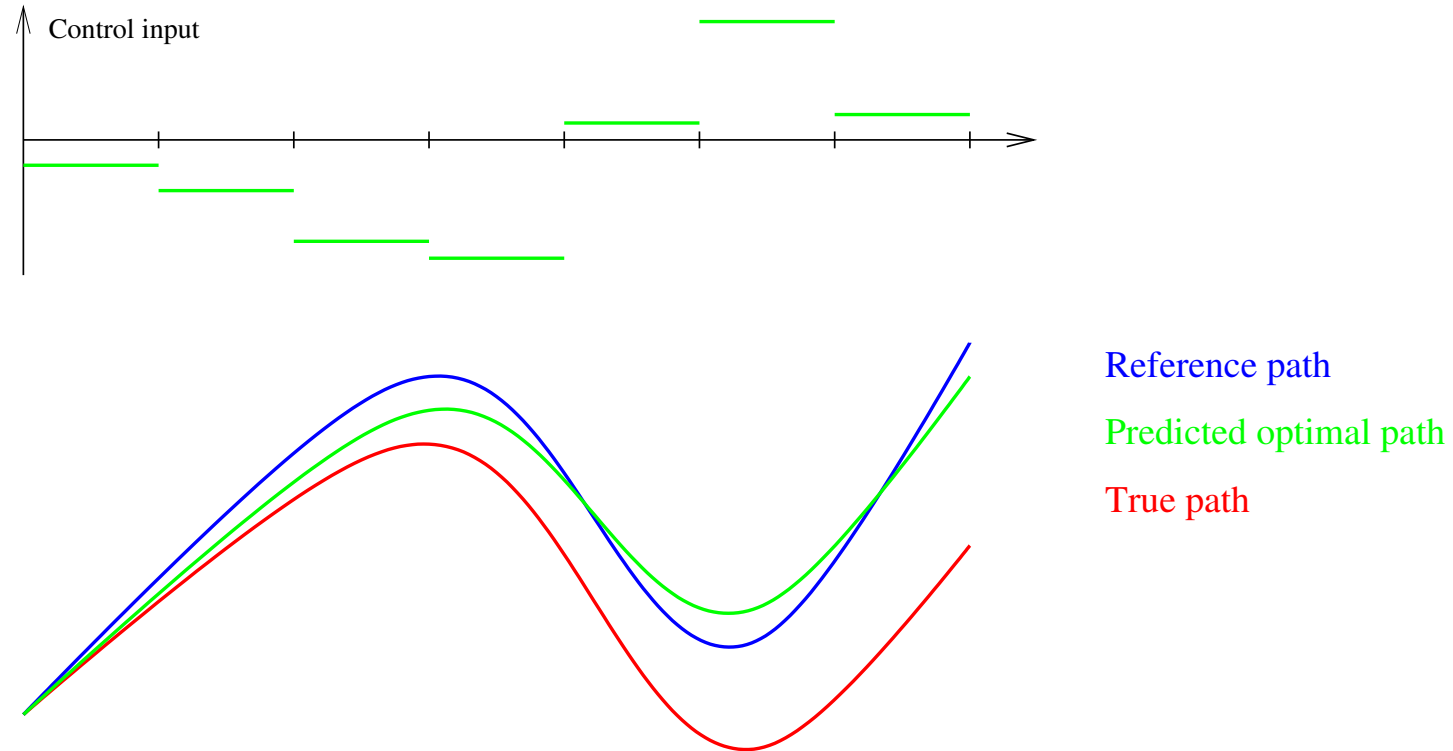
$$Q = \begin{pmatrix} q_1 & 0 \\ 0 & q_2 \end{pmatrix},$$

the parameters get intuitive meaning and the cost-function becomes

$$\sum_{t=0}^{\infty} q_1 d_t^2 + q_2 \theta_{e,t}^2 + R \kappa_t^2$$

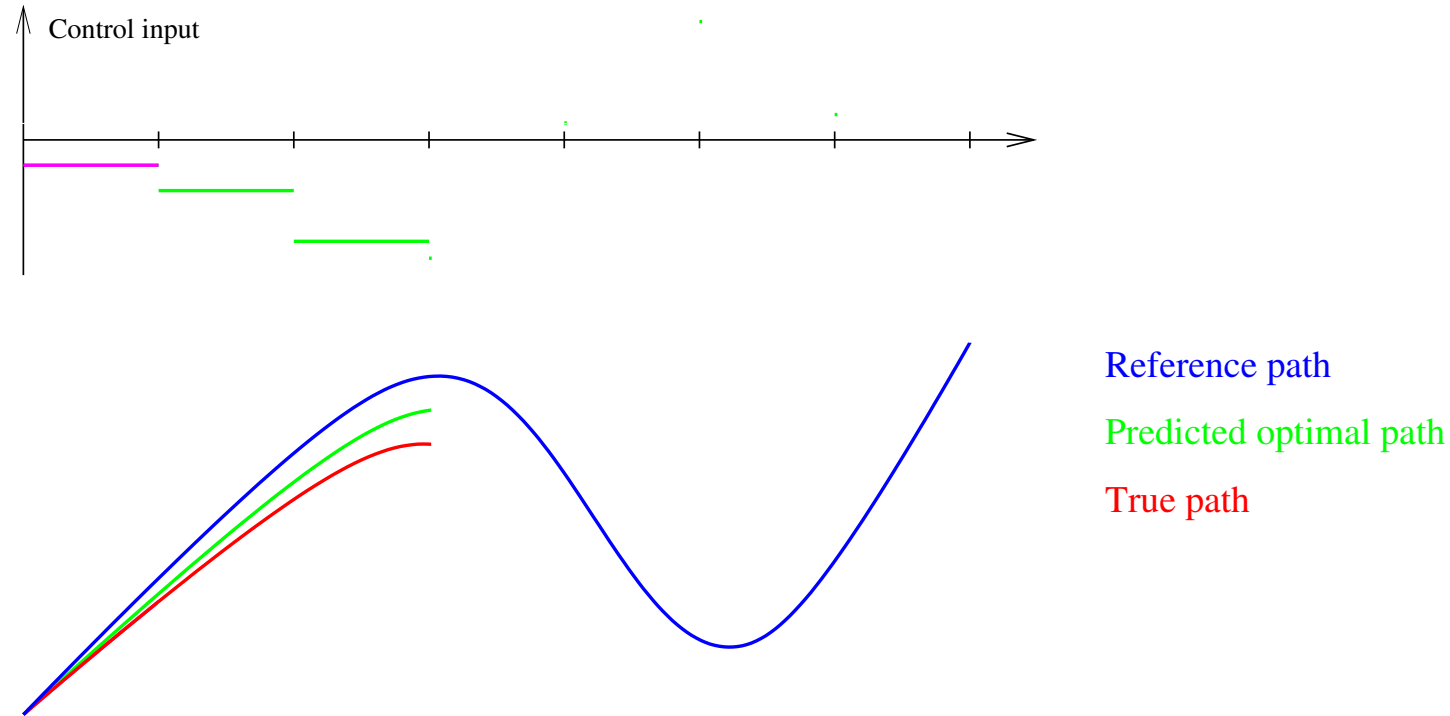
where d_t , $\theta_{e,t}$, and κ_t are state variables and inputs in the linearized problem.

Path Tracking with Open Loop Control



$$\min \sum_{t=1}^N q_1 d_t^2 + q_2 \theta_{e,t}^2 + R \kappa_t^2$$

MPC step 1

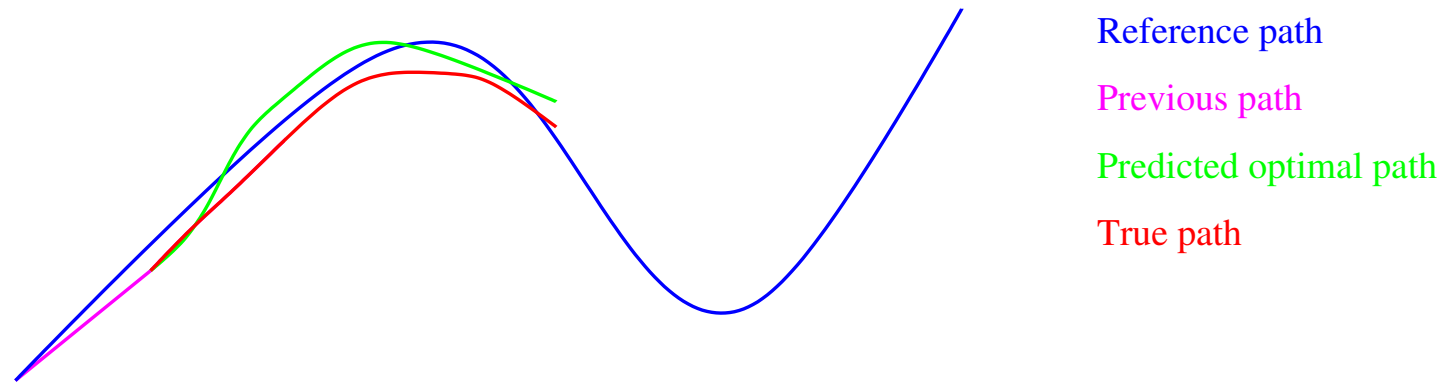
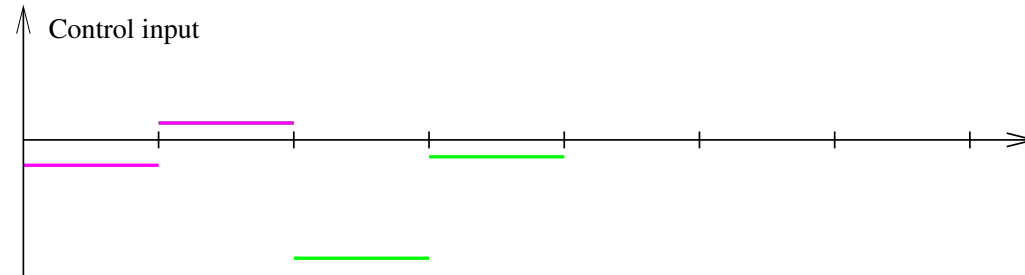


$$\min \sum_{t=1}^3 q_1 d_t^2 + q_2 \theta_{e,t}^2 + R \kappa_t^2$$

Compute the optimal path for a shorter prediction horizon
(in this example, three time steps)

Use only the first control (t=1) and forget the others

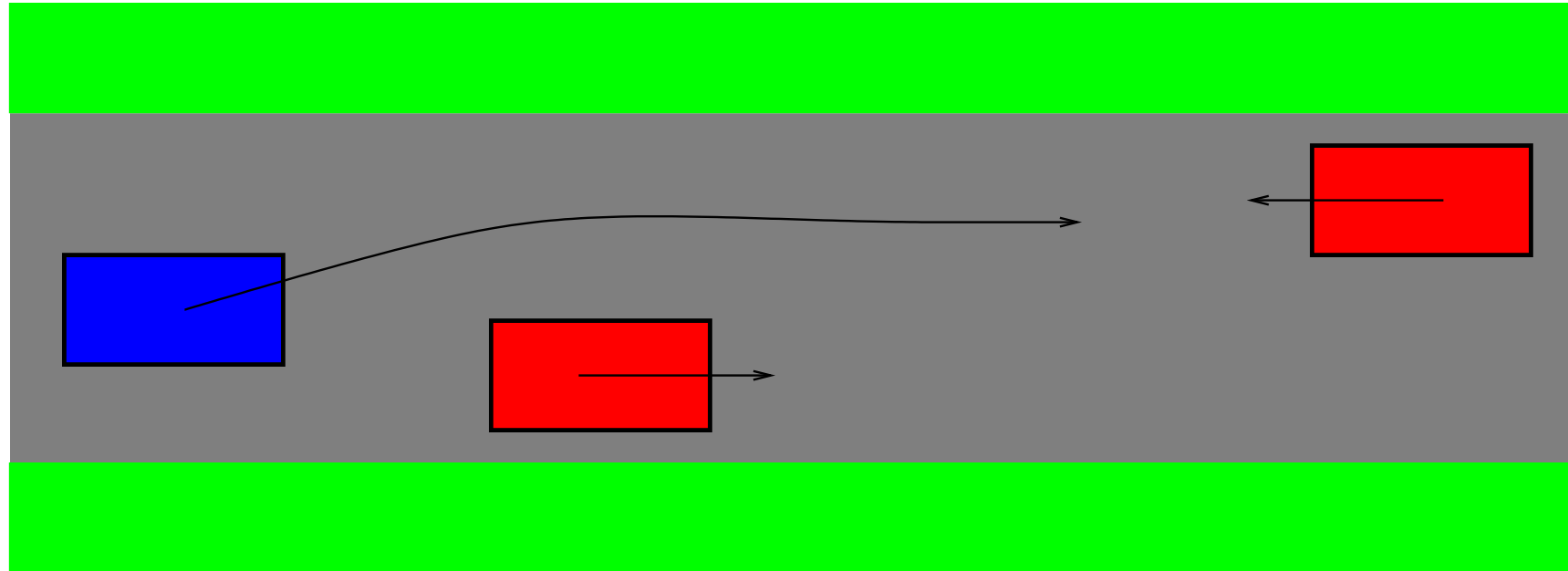
MPC step 2



$$\min \sum_{t=2}^4 q_1 d_t^2 + q_2 \theta_{e,t}^2 + R \kappa_t^2$$

Repeat the procedure on the previous slide to compute the control signal at t=2

Overtaking with too short horizon



Too short prediction horizon $\approx$ Near-sighted
(I don't think I have to explain the drawback with this)

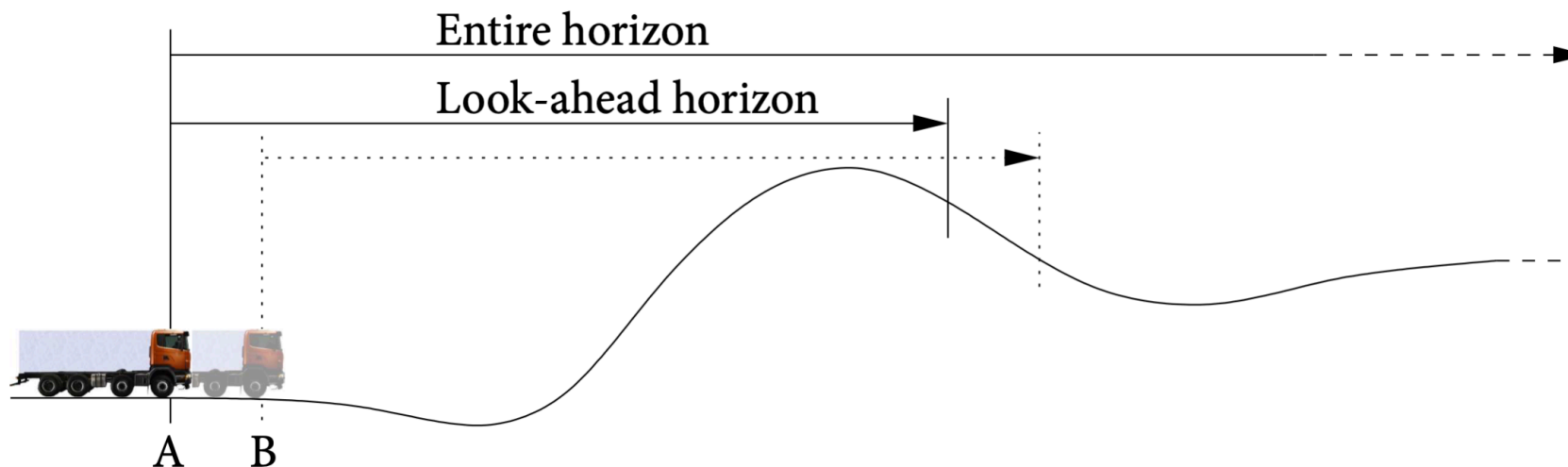
Cornering with too short horizon

9



Similar problem as on the previous slide
You see the corner to late

Look Ahead Controller with too short horizon



$\min\{\alpha M + \beta T\}$, where M is the consumed fuel/energy over the prediction horizon, and T is the corresponding time. This is a multi-objective optimization problem, and the weights α and β represents the trade-off.

If the prediction horizon (look-ahead horizon) is too short, the truck might start accelerating too late and will not be able to reach the top of the hill.

Outline of the Lecture

- **Fundamentals of model predictive control (MPC).**
- **Trajectory tracking** for autonomous vehicles using **MPC.**
- **Path following** for autonomous vehicles using **MPC.**
- **Tools and libraries for optimization and MPC.**

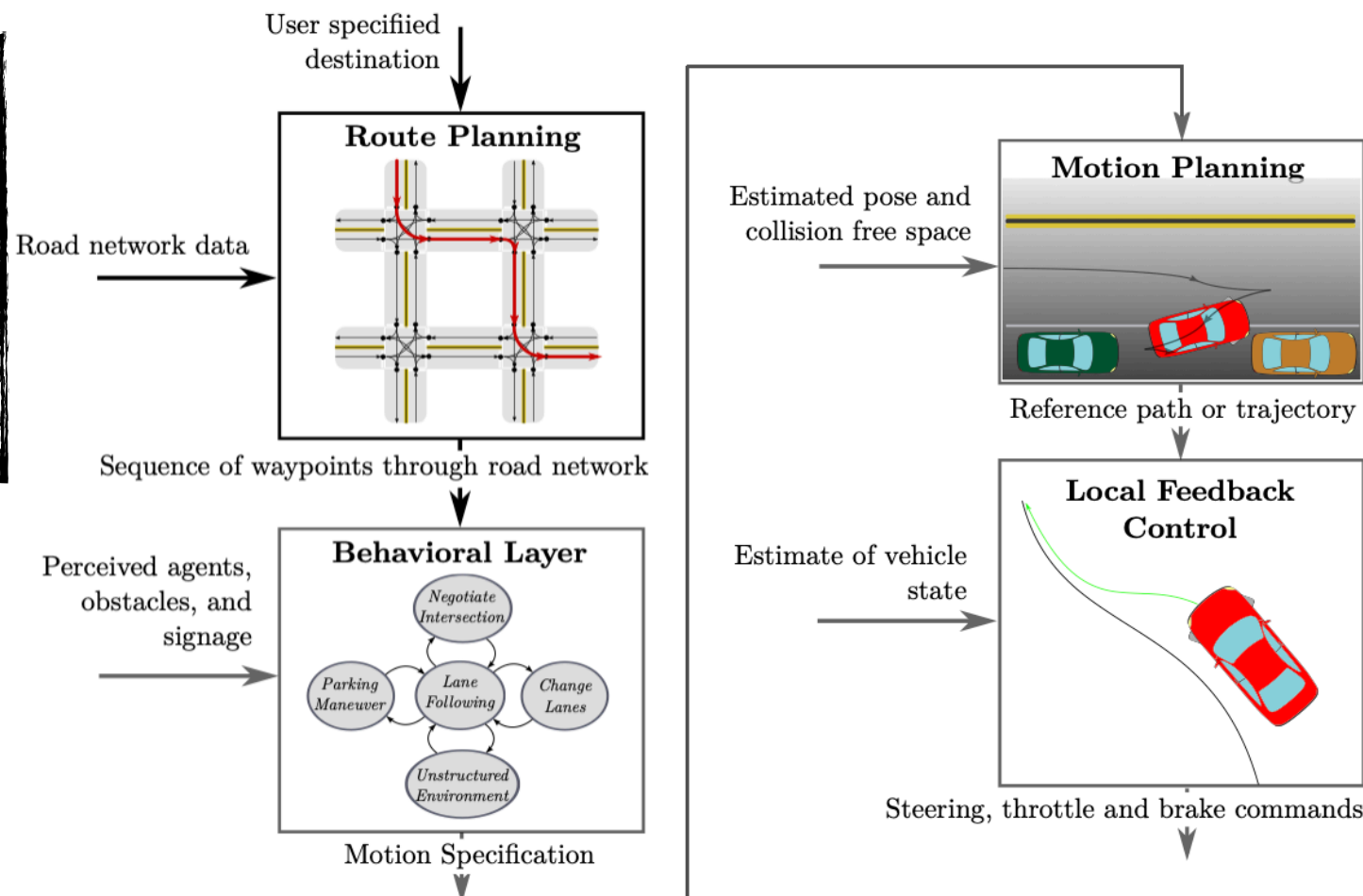
Literature Reading

The following book and article sections are the main reading material for this lecture. References to further reading are provided throughout the slides and at the end of the lecture slides.

- Chapter 1 in Rawlings, J. B., D. Q. Mayne, & M. Diehl: *Model Predictive Control: Theory, Computation, and Design*. 2nd Edition. Nob Hill Publishing, 2017.
- Section V-C in Paden, B., Čáp, M., Yong, S. Z., Yershov, D., & Frazzoli, E: "A survey of motion planning and control techniques for self-driving urban vehicles". *IEEE Transactions on Intelligent Vehicles*, 1(1), 33-55, 2016.

Context in the Architecture for an Autonomous Vehicle

Model predictive control is possible to apply at several different layers in the architecture for different tasks.

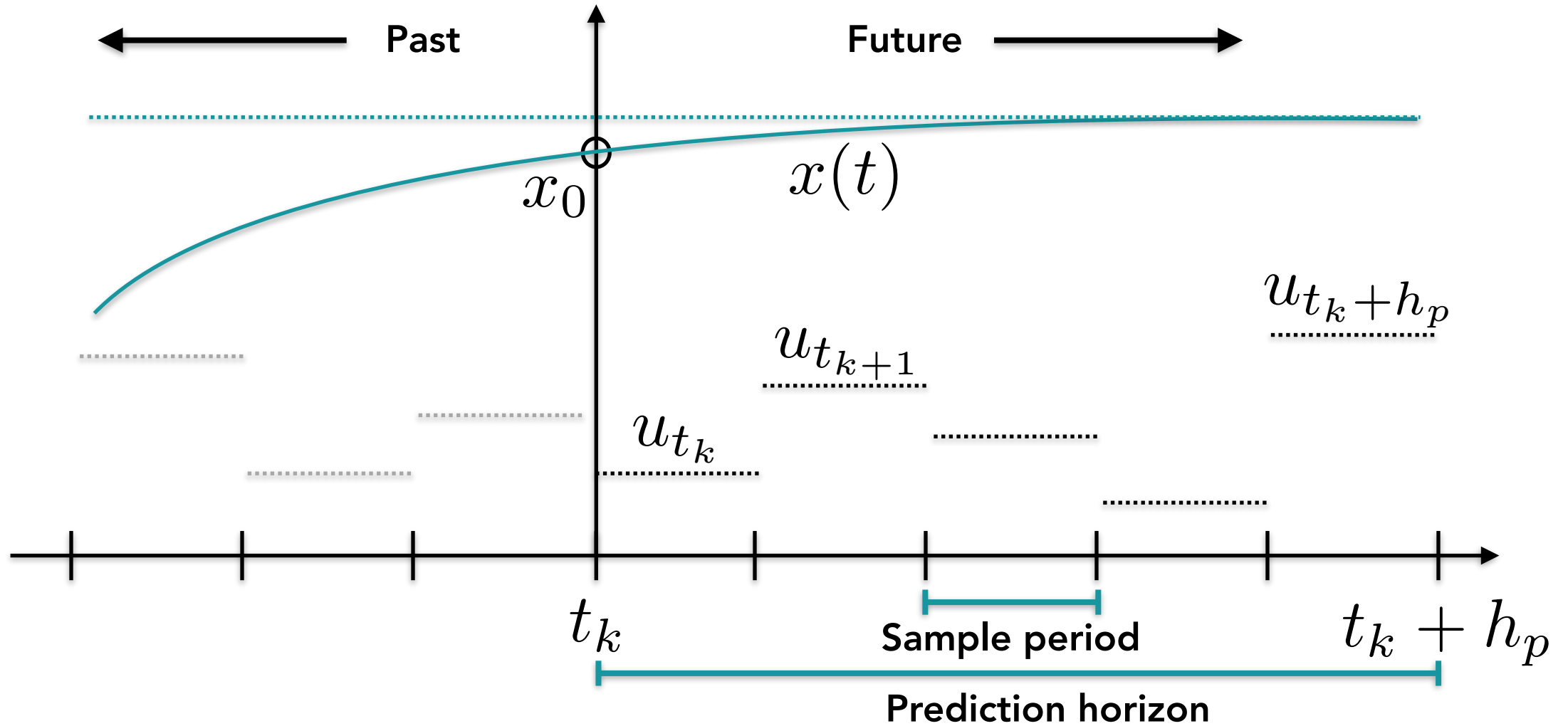


Overview of Control Methods for Autonomous Vehicles

	Controller		Model	Stability	Time Complexity	Comments/Assumptions
Lecture 6	Pure Pursuit	(V-A1)	Kinematic	LES [*] to ref. path	$O(n)^*$	No path curvature
	Rear wheel based feedback	(V-A2)	Kinematic	LES [*] to ref. path	$O(n)^*$	$C^2(\mathbb{R}^n)$ ref. paths
	Front wheel based feedback	(V-A3)	Kinematic	LES [*] to ref. path	$O(n)^*$	$C^1(\mathbb{R}^n)$ ref. paths; Forward driving only
	Feedback linearization	(V-B2)	Steering rate controlled kinematic	LES [*] to ref. traj.	$O(1)$	$C^1(\mathbb{R}^n)$ ref. traj.; Forward driving only
Lecture 6	Control Lyapunov design	(V-B1)	Kinematic	LES [*] to ref. traj.	$O(1)$	Stable for constant path curvature and velocity
This lecture	Linear MPC	(V-C)	$C^1(\mathbb{R}^n \times \mathbb{R}^m)$ model [#]	LES [*] to ref. or path	$O\left(\sqrt{N} \ln\left(\frac{N}{\varepsilon}\right)\right)^\dagger$	Stability depends on horizon length
This lecture	Nonlinear MPC	(V-C)	$C^1(\mathbb{R}^n \times \mathbb{R}^m)$ model [#]	Not guaranteed	$O\left(\frac{1}{\varepsilon}\right)^\ddagger$	Works well in practice

Fundamentals of Model Predictive Control

Model Predictive Control (MPC)



Key Concepts in MPC (1/2)

- A **model** of the system is used for **prediction**.
- The length of the finite horizon is called the **prediction horizon**.
- The **control horizon** is the **subset of the prediction horizon** over which the control inputs are allowed to vary.
 - Could also be the **full set**, i.e., the horizons coincide.

Key Concepts in MPC (2/2)

- **Cost function** describing desired **objective** and **constraints** on control inputs and states.
- User-specified **weights** often used to trade **different components** to each other in the **cost function** of the MPC formulation.

Recall Optimization Problem from Lecture 5

- **Optimization problem** over time horizon $[0, T]$, where T possibly is a free optimization variable:

minimize $\int_0^T L(x(t), u(t)) dt + \Gamma(x(T))$

subject to $x(0) = x_0, \dot{x}(t) = f(t, x(t), u(t)),$

$x(t) \in \mathbb{X}, u(t) \in \mathbb{U}, x(T) \in \mathbb{X}_T, t \in [0, T]$

Diagram labels and arrows:

- Lagrange integrand** points to $L(x(t), u(t))$
- Mayer term** points to $\Gamma(x(T))$
- Initial conditions** points to $x(0) = x_0$
- System dynamics** points to $\dot{x}(t) = f(t, x(t), u(t))$
- State and control constraints** points to $x(t) \in \mathbb{X}, u(t) \in \mathbb{U}$
- Terminal constraints** points to $x(T) \in \mathbb{X}_T$

A General MPC Formulation

- Solve **optimization problem** over the **time horizon** $[t_k, t_k + h_p]$ at selected time instants t_k , $k = 0, 1, 2, \dots$, with h_p the prediction horizon:

$$\begin{aligned} &\text{minimize} && \int_{t_k}^{t_k+h_p} L(x(t), u(t)) \, dt + \Gamma(x(t_k + h_p)) \\ &\text{subject to} && x(t_k) = x_0, \quad \dot{x}(t) = f(t, x(t), u(t)), \\ &&& x(t) \in \mathbb{X}, \quad u(t) \in \mathbb{U}, \quad x(t_k + h_p) \in \mathbb{X}_{t_k+h_p}, \quad t \in [t_k, t_k + h_p] \end{aligned}$$

- The **computed control inputs** are applied until next time step t_{k+1} .
- The **current states** x_0 are updated at each new iteration.

A QP MPC Formulation (1/2)

- Recall the **LQR control design** from Lecture 6. Assume a **linear model**:

$$\dot{x}(t) = Ax(t) + Bu(t)$$

- Choose the **cost function** as the following:

$$L(x(t), u(t)) = x^T(t)Qx(t) + u(t)^T Ru(t)$$
$$\Gamma(x(t_k + h_p)) = x(t_k + h_p)^T P_f x(t_k + h_p)$$

A QP MPC Formulation (2/2)

- The possibly time-varying **weight matrices** Q, R, P_f are positive (semi-)definite and are specified by the user.
 - Trade **different objectives** to each other.
- With all remaining constraints linear or affine, the problem is a **quadratic program (QP)**, i.e., a convex optimization problem.

Model Considerations in MPC

- The model equations of the system appear as a **constraint**, therefore imply properties of the resulting **optimization problem**.
- Recall the discussion on **convex vs. non-convex optimization** from Lecture 5.
- **Non-linear models** possible to include in the **MPC formulation**.
- **Linearizing a non-linear model** along a **nominal/reference trajectory** leads to a linear time-varying (LTV) system:

$$\dot{x}(t) = A(t)x(t) + B(t)u(t)$$

Hard vs. Soft Constraints

- Explicit constraint that needs to be fulfilled exactly – **hard constraint**.
- **Penalty in cost function** with certain weight – **soft constraint** (fulfil it as good as possible, given other objectives).
 - Could be beneficial in real-time applications.

Hard vs. Soft Constraints – Example

- Assume that the speed v of an autonomous car should be kept close to a **desired reference velocity** v_{ref} .
- A **soft constraint** can be formulated as a component in the **cost function** as:

$$\int_{t_k}^{t_k+h_p} (v - v_{\text{ref}})^2 dt$$

- A **hard constraint** can be formulated as an **explicit constraint** as:

$$v_{\text{ref}} - \varepsilon \leq v \leq v_{\text{ref}} + \varepsilon$$

Discretization of MPC Problem

- **Continuous-time MPC problems** need to be transformed to a **finite-dimensional optimization problem** (a non-linear program, **NLP**) for solution in a computer.
- Recall the methods from Lecture 5:
 - Direct **multiple shooting** or **collocation**.
 - **Discretization** of **system dynamics** and **cost function**, with polynomial approximation of control inputs and possibly states.

Computational Considerations

- A **longer prediction horizon** h_p implies a larger optimization problem, and thus higher computational cost.
- Also the **length of the sampling period** $\Delta t = t_{k+1} - t_k$, i.e., how often the MPC problem is solved, affects the computational cost.
- A **control horizon shorter than the prediction horizon** can be used to save computational resources.

Stability of MPC

- **Length of prediction horizon** important in practice.
 - Sufficiently long horizon avoids short-term optimization.
- Choice of **cost** or **constraint** at **end of prediction horizon**.
 - Compare with having a sufficiently good **cost-to-go estimate**.
- **Extensive theory** exists for different formulations, see references for further details.

Why MPC for Autonomous Vehicles?

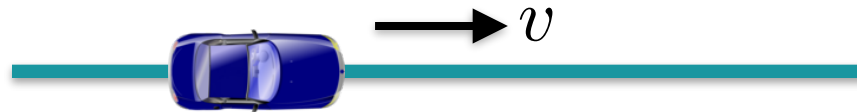
- Constantly **changing environment** with **inherent uncertainty** typical for autonomous vehicles.
- Different **sensor systems** provide updated information periodically (**situation awareness**).
- MPC offers **re-planning capabilities** combined with **optimal feedback control**.



MPC for Autonomous Trajectory Tracking

Trajectory Tracking vs. Path Following

- Recall the difference between a **path** (curve in space) and a **trajectory** (time-parameterized, velocity profile included).
- **MPC** can be formulated both for **trajectory tracking** and **path-following** applications.
- Consider the **example** of **driving along a straight line**.
 - Driving exactly at the straight line with a lower speed than according to the reference trajectory, gives a large trajectory error but no path error.



Problem Formulation for Trajectory Tracking

- Assume that we have **reference values** x_{ref} for the **states** that should be tracked.
- An **MPC cost function** for trajectory tracking could then be:

$$\int_{t_k}^{t_k + h_p} \left\{ (x(t) - x_{\text{ref}}(t))^T Q (x(t) - x_{\text{ref}}(t)) + u(t)^T R u(t) \right\} dt$$

Example Constraints in Trajectory Following

- A key feature of MPC is that **constraints** can be included in the formulation and thus considered in the controller.

- **Linear constraints** on **inputs** and **states** formulated as:

$$u_{\min} \leq u(t) \leq u_{\max}$$

$$x_{\min} \leq x(t) \leq x_{\max}$$

- Input can be **acceleration** and states could be **position** and **velocity** for an autonomous vehicle, thus specifying, e.g., **maximum forward acceleration** or **not allowed areas in geometric space**.

MPC for Kinematic Path Following

Path Following for Kinematic Model

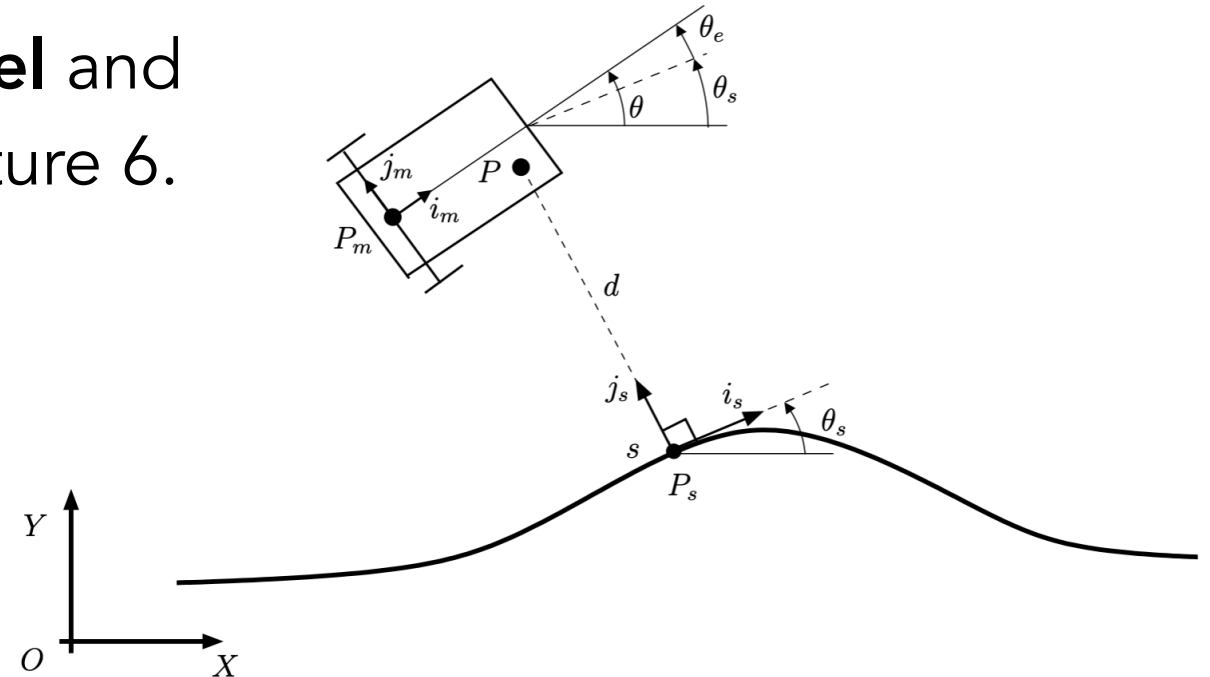
- Recall the **kinematic motion model** and **path-following** problem from Lecture 6.

$$\dot{s} = \frac{v}{1 - dc(s)} \cos(\theta_e),$$

$$\dot{d} = v \sin(\theta_e),$$

$$\dot{\theta}_e = vu - \dot{sc}(s)$$

- States $x = (d \quad \theta_e)$ and system dynamics $f(x, u) = \begin{pmatrix} v \sin(\theta_e) \\ vu - \dot{sc}(s) \end{pmatrix}$



MPC Formulation for Kinematic Path Following (1/2)

- An **MPC problem** for **path following**, where the objective is to achieve that $x \rightarrow 0$ (i.e., no path deviations).
- With the **weights** $Q = \text{diag}(q_1, q_2)$, $R = r$, the MPC problem for **path following** can be formulated as:

$$\begin{aligned} &\text{minimize} && \int_{t_k}^{t_k+h_p} q_1 d^2 + q_2 \theta_e^2 + r u^2 dt \\ &\text{subject to} && x(t_k) = x_0, \quad \dot{x}(t) = f(x(t), u(t)), \\ &&& u(t) \in \mathbb{U}, \quad t \in [t_k, t_k + h_p] \end{aligned}$$

MPC Formulation for Kinematic Path Following (2/2)

- The **constraint** on the control input can be determined based on **steering-angle limitations** as:

$$\mathbb{U} = \{u \mid 1/L \tan(\delta_{\min}) \leq u \leq 1/L \tan(\delta_{\max})\}$$

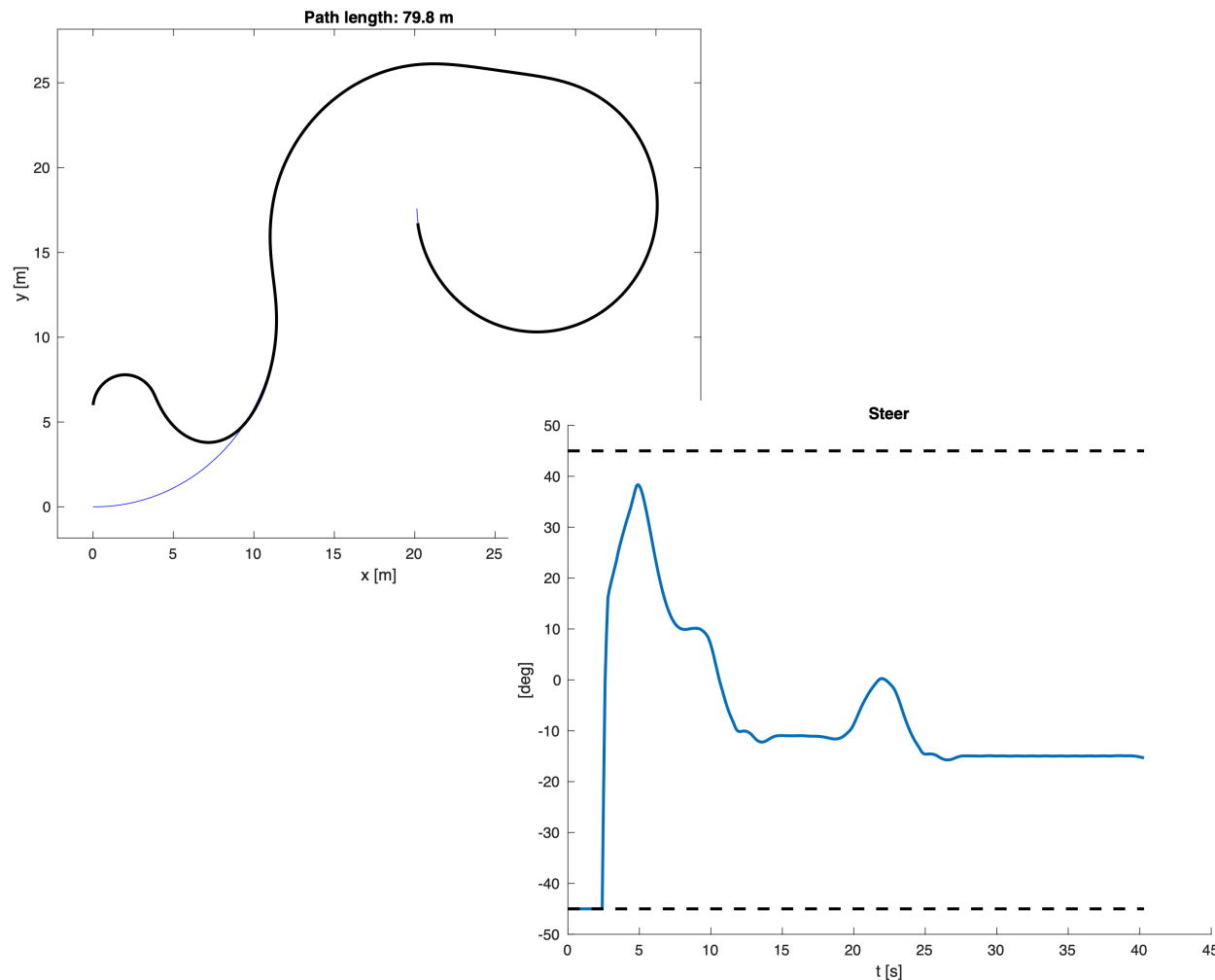
- Compared to the LQR approach, with **MPC** a **finite-horizon optimization problem** is instead solved at each sample instant, considering both the **non-linear vehicle model** and **constraints** on the input.
 - Results in a **computationally more expensive** controller.

Example: Extra Assignment for Hand-in 3 (1/2)

- Part of extra assignment for **Hand-in Exercise 3** to implement this **path-following MPC**.
- **Linearization** of the non-linear kinematic single-track model at each MPC update step.
 - **Quadratic cost function** and **linear constraints**, leads to **convex optimization problem** (efficient solvers exist).

Example: Extra Assignment for Hand-in 3 (2/2)

- Application on an **example path** from Hand-in Exercise 3.
- **Constraints on control signal** (steering angle).
- Choices of **prediction horizon** and **sample period**.
- **Multiple shooting** with **explicit Runge-Kutta** method of fourth order for **discretization** of vehicle motion equations (see Lecture 5).



MPC for Autonomous Path Following



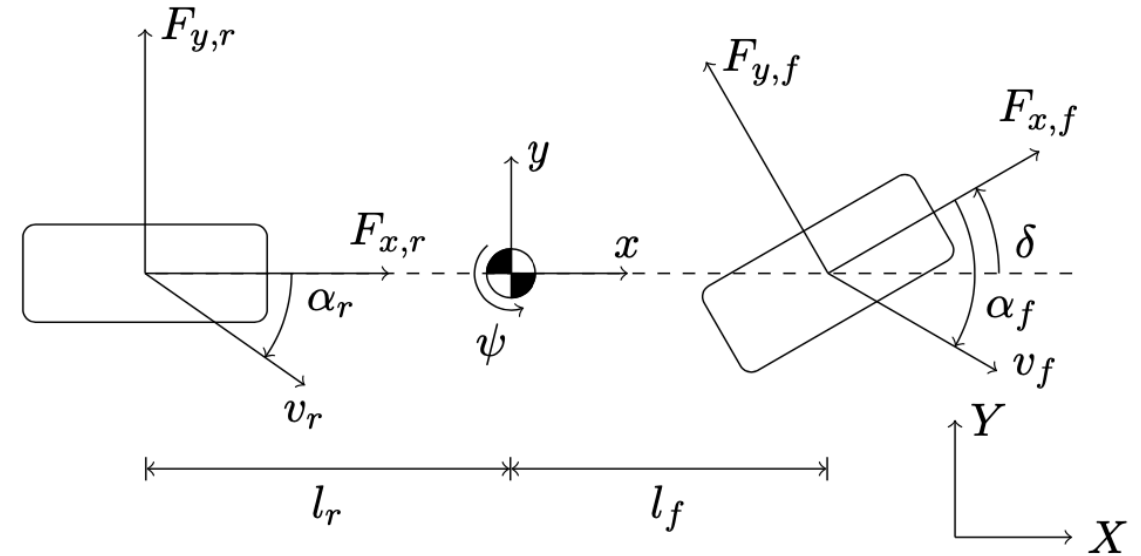
Faster Driving

Problem Formulation

- The task is to **follow a desired path** (position and orientation) and a nominal **velocity profile**.
- To account for **dynamic effects** of a ground vehicle, a **single-track model** with a **linear tire–road interaction model** is used.
 - More **complex model** than the kinematic model in the previous example, beneficial for **higher velocities** and in **more advanced maneuvers**.
- Design an **MPC** (objective and constraints) for this task.

Dynamic Vehicle Model (1/4)

- Consider the **single-track vehicle model** (lumped wheels on each axle).
- Front steering and front-wheel driven.**



$$m(\dot{v}_x - v_y\dot{\psi}) = F_{x,f} \cos(\delta) + F_{x,r} - F_{y,f} \sin(\delta),$$

$$m(\dot{v}_y + v_x\dot{\psi}) = F_{y,f} \cos(\delta) + F_{y,r} + F_{x,f} \sin(\delta),$$

$$I_Z\ddot{\psi} = l_f F_{y,f} \cos(\delta) - l_r F_{y,r} + l_f F_{x,f} \sin(\delta)$$

Dynamic Vehicle Model (2/4)

- The **slip angles** (angle between the velocity vector and the direction of the wheel) are given by:

$$\alpha_f = -\arctan\left(\frac{v_{f,y}}{v_{f,x}}\right), \quad \alpha_r = -\arctan\left(\frac{v_{r,y}}{v_{r,x}}\right)$$

- A low-complexity **tire-road interaction model** for normal driving with linear tire stiffnesses is adopted:

$$F_{y,f} = C_{\alpha,f}\alpha_f, \quad F_{y,r} = C_{\alpha,r}\alpha_r$$

Dynamic Vehicle Model (3/4)

- A **path parameter** is introduced to describe the traversal along the reference path (*traversal computed in the MPC*):

$$\dot{s} = \frac{ds}{dt}$$

- The **vehicle position** in the **global coordinate frame** is obtained by integration of the quantities:

$$\begin{pmatrix} \dot{p}_X \\ \dot{p}_Y \end{pmatrix} = \begin{pmatrix} \cos(\psi) & -\sin(\psi) \\ \sin(\psi) & \cos(\psi) \end{pmatrix} \begin{pmatrix} v_x \\ v_y \end{pmatrix}$$

Dynamic Vehicle Model (4/4)

- Collect the **states** and the **control inputs** in the vectors:

$$x(t) = (p_X \quad p_Y \quad \psi \quad v_x \quad v_y \quad \dot{\psi} \quad s)^T$$

$$u(t) = (\delta \quad F_{x,f} \quad F_{x,r} \quad \dot{s})^T$$

- With these variables, the **vehicle dynamics** can be written as an **explicit ordinary differential equation system** as:

$$\dot{x}(t) = f_{\text{car}}(x(t), u(t))$$

MPC Problem (1/4)

- **Reference values** for the vehicle consist of **position** in global coordinate frame, **longitudinal velocity**, and **yaw orientation**.
- Introduce the **vector of reference values** as:

$$\begin{pmatrix} p_{X,\text{ref}} & p_{Y,\text{ref}} & v_{x,\text{ref}} & \psi_{\text{ref}} \end{pmatrix}^T$$

MPC Problem (2/4)

- Introduce a function that describes the **error quantities** that the controller should **drive to zero** (*reference values as function of the path parameter s*):

$$\Phi(x(t)) = \begin{pmatrix} p_{X,\text{ref}}(s) - p_X(t) \\ p_{Y,\text{ref}}(s) - p_Y(t) \\ e_{\perp}(t, s) \\ \psi_{\text{ref}}(s) - \psi(t) \\ v_{x,\text{ref}}(s) - v_x(t) \end{pmatrix}$$

- The **orthogonal projection** of the path error is:

$$e_{\perp}(t, s) = \sin(\psi_{\text{ref}})(p_{X,\text{ref}}(s) - p_X(t)) - \cos(\psi_{\text{ref}})(p_{Y,\text{ref}}(s) - p_Y(t))$$

MPC Problem (3/4)

- The **finite-horizon MPC problem** to be solved at each sample instant can then be stated as:

$$\begin{aligned} &\text{minimize} && \int_{t_k}^{t_k+h_p} \Phi^T Q \Phi + u^T R u \, dt \\ &\text{subject to} && x(t_k) = x_0, \quad \dot{x}(t) = f_{\text{car}}(x(t), u(t)), \\ &&& x(t) \in \mathbb{X}, \quad u(t) \in \mathbb{U}, \quad t \in [t_k, t_k + h_p] \end{aligned}$$

- The weights Q , R are used to **trade different objectives** in the cost function to each other.

MPC Problem (4/4)

- There is **large flexibility** in terms of the choice of **constraints**.
- There are typically constraints on the **steering angle** and the **driving** and **braking forces**.
- Update of **path parameter** also constrained.
- **Geometric constraints** also possible.

$$\delta_{\min} \leq \delta \leq \delta_{\max},$$

$$F_{x,f,\min} \leq F_{x,f} \leq F_{x,f,\max},$$

$$F_{x,r,\min} \leq F_{x,r} \leq F_{x,r,\max},$$

$$1 - s_{\Delta} \leq \dot{s} \leq 1 + s_{\Delta}$$

Solution of the MPC Problem

- At **each sample instant**, the finite-horizon **MPC problem is solved** with updated state measurements (or often, in practice *state estimates*).
- The model is non-linear, and thus the problem is a **non-linear MPC** (NMPC).
- **Numerical solution** of the **optimization problem** required.

Solving MPC Optimization Problems

- **A large number of optimization problems** are solved in sequence online in MPC.
- Often there are **real-time computational constraints** for an MPC controller executing **online**.
- How can the associated **optimization problem** be **solved efficiently** and the **special structure** of it be utilized?

Solving MPC Optimization Problems Using RTI

- One proposed method is the **Real-Time Iteration (RTI)** scheme:
 - Multiple shooting for discretization of continuous dynamics.
 - Sequential quadratic programming (**SQP**), using one single Newton step, and an effective **warm starting** by using the state and control input trajectories from the previous MPC iteration.
- Separation of computations in ***preparation phase*** and ***feedback response phase*** (highly important in real-time implementations).

Tools and Libraries for Numerical Optimal Control and Model Predictive Control

CasADi



- Open-source tool for **numerical optimization and optimal control**.
- Includes efficient algorithms for **automatic differentiation** and computing Jacobians and Hessians in forward and backward mode.
- **Interfaces** to various NLP solvers and numerical integrators for ODEs/DAEs.
- Homepage: <https://web.casadi.org>

IPOPT

Andreas Wächter · Lorenz T. Biegler

On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming

- Software package for **solving large-scale NLPs**.
- Implements a **primal-dual interior-point method** for finding local minima to the NLP.
- **Scaling** of model equations for numerical stability.
- Relies on external software packages for solving the **inherent linear equation systems** (e.g., MA27 and MA57 from HSL Mathematical Software Library).
- Homepage: <https://github.com/coin-or/lpopt>

YOP

- YOP – Yet Another Optimal Control Problem Parser.
- **Matlab toolbox** for **numerical optimal control**, with an interface to CasADi for access to **integrators** and **solvers of nonlinear programs**.
- Developed at Div. Vehicular Systems at Linköping University.
- Several **examples** available on toolbox homepage: <https://www.yoptimalcontrol.se/index.html>

References and Further Reading

References and Further Reading (1/2)

All the following books and articles are not part of the reading assignments for the course, but cover the topics studied during this lecture in more detail.

- Andersson, J., J. Gillis, G. Horn, and J. B. Rawlings, & M. Diehl: "CasADi—A software framework for nonlinear optimization and optimal control". *Mathematical Programming Computation*, 2018.
- Berntorp, K., Quirynen, R., Uno, T., & Di Cairano, S: "Trajectory tracking for autonomous vehicles on varying road surfaces by friction-adaptive nonlinear model predictive control". *Vehicle System Dynamics*, 58(5), 705-725, 2020.
- Bertsekas, D. P: *Reinforcement learning and optimal control*. Belmont, MA: Athena Scientific, 2019.
- Di Cairano, S., Kalabić, U. V., & Berntorp, K: "Vehicle tracking control on piecewise-clothoidal trajectories by MPC with guaranteed error bounds". In IEEE Conference on Decision and Control (CDC), 709-714, 2016.
- Diehl, M., Bock, H. G., & Schlöder, J. P: "A real-time iteration scheme for nonlinear optimization in optimal feedback control". *SIAM Journal on Control and Optimization*, 43(5), 1714-1736, 2005.

References and Further Reading (1/2)

All the following books and articles are not part of the reading assignments for the course, but cover the topics studied during this lecture in more detail.

- Gros, S., Zanon, M., Quirynen, R., Bemporad, A., & Diehl, M: "From linear to nonlinear MPC: Bridging the gap via the real-time iteration". *International Journal of Control*, 93(1), 62-80, 2020.
- Faulwasser, T., & Findeisen, R: "Nonlinear model predictive control for constrained output path following". *IEEE Transactions on Automatic Control*, 61(4), 1026-1039, 2015.
- Hellström, E., Ivarsson, M., Åslund, J., & Nielsen, L: "Look-ahead control for heavy trucks to minimize trip time and fuel consumption". *Control Engineering Practice*, 17(2), 245-254, 2009.
- Paden, B., Čáp, M., Yong, S. Z., Yershov, D., & Frazzoli, E: "A survey of motion planning and control techniques for self-driving urban vehicles". *IEEE Transactions on Intelligent Vehicles*, 1(1), 33-55, 2016.
- Rao, C. V., Rawlings, J. B., & Mayne, D. Q: "Constrained state estimation for nonlinear discrete-time systems: Stability and moving horizon approximations". *IEEE Transactions on Automatic Control*, 48(2), 246-258, 2003.
- Rawlings, J. B., D. Q. Mayne, & M. Diehl: *Model Predictive Control: Theory, Computation, and Design*. 2nd Edition. Nob Hill Publishing, 2017.

Closed Loop Rapidly-Exploring Random Tree (CL-RRT)

Material from the previous lectures

Closed Loop Rapidly-Exploring Random Tree (CL-RRT)

Today I will present the method Closed Loop Rapidly-Exploring Random Tree. The presentation will be based on the following material from previous lectures:

- Kinematic Model (lecture 3)
- Dubins Car (lecture 3)
- Rapidly-exploring random tree (lecture 4)
- Pure-Pursuit Control (lecture 6)

The main reference is the paper:

[Real-Time Motion Planning With Applications to Autonomous Urban Driving, Kuwata et.al., IEEE TRANSACTIONS ON CONTROL SYSTEMS TECHNOLOGY, VOL. 17, NO. 5, SEPTEMBER 2009](#)

Kinematic Model (Lecture 3)

Example of a kinematic model:

$$\dot{x} = v \cos \theta$$

$$\dot{y} = v \sin \theta$$

$$\dot{\theta} = v \tan \delta / l$$

$$\dot{\delta} = u_1$$

$$\dot{v} = a$$

$$\dot{a} = u_2$$

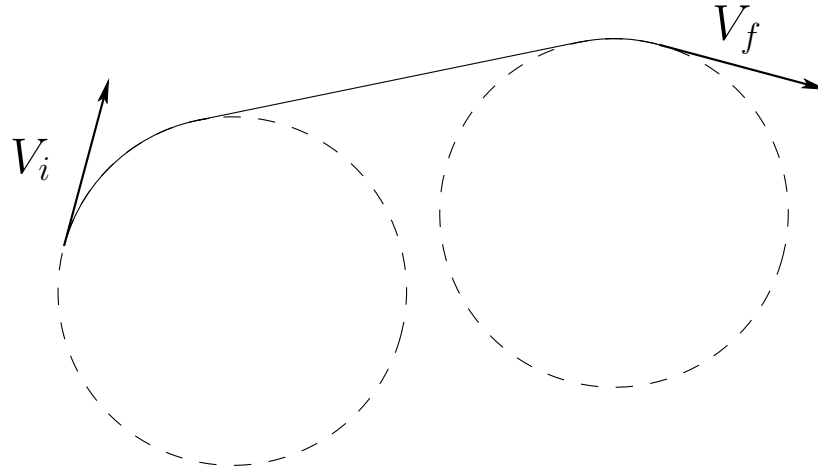
$$\delta \in [-\delta_{max}, \delta_{max}]$$

$$u_1 \in [-\dot{\delta}_{max}, \dot{\delta}_{max}]$$

Note that the steering angle and acceleration are states.
This gives a smoother trajectory.

Dubins Car (Lecture 3)

Problem: Find the shortest path between two points with the orientation specified at the initial and final point, and the turning radius limited from below

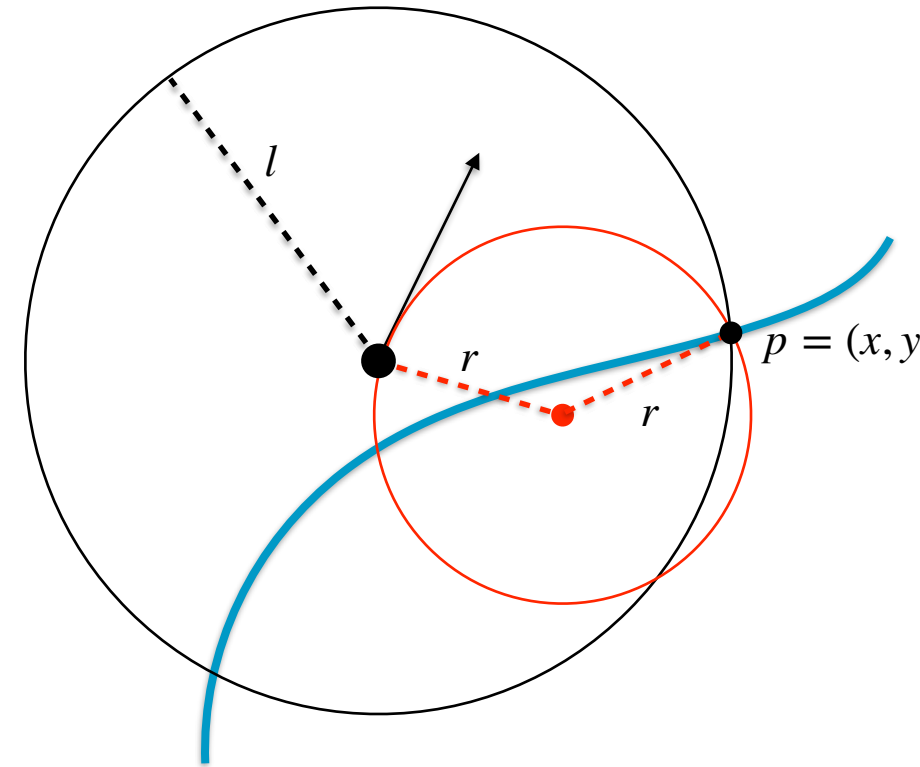


Solution: The optimal solution consists of segments with minimal radius R_{min} and straight lines.

Pure-pursuit control (Lecture 6)

- A simple control technique to compute the arc needed for a robot to get back on path
- With a look-ahead horizon, l , find point $p = (x, y)$ on the path to aim for
- Compute the turning radius r to get there
- For a single-track robot, this corresponds to a steering angle

$$\frac{1}{L} \tan \delta = \frac{1}{r}$$



Basic Version of RRT (Lecture 4)

Algorithm 1: RRT w/o. Differential Constraints

```

1   $\mathcal{V} \leftarrow \{q_I\}, \mathcal{E} \leftarrow \emptyset;$ 
2  for  $i = 1, \dots, N$  do:
3       $q_{\text{rand}} \leftarrow \text{Sample};$ 
4       $q_{\text{nearest}} \leftarrow \text{Nearest}(\mathcal{G} = (\mathcal{V}, \mathcal{E}), q_{\text{rand}});$ 
5       $q_{\text{new}} \leftarrow \text{Steer}(q_{\text{nearest}}, q_{\text{rand}});$ 
6      if  $\text{ObstacleFree}(q_{\text{nearest}}, q_{\text{new}})$  then
7           $\mathcal{V} \leftarrow \mathcal{V} \cup \{q_{\text{new}}\};$ 
8           $\mathcal{E} \leftarrow \mathcal{E} \cup \{(q_{\text{nearest}}, q_{\text{new}})\};$ 
9  return  $\mathcal{G} = (\mathcal{V}, \mathcal{E});$ 

```

LaValle, S. M., & J. J. Kuffner Jr.: "Randomized kinodynamic planning". *The International Journal of Robotics Research*, 20(5), 378-400, 2001.

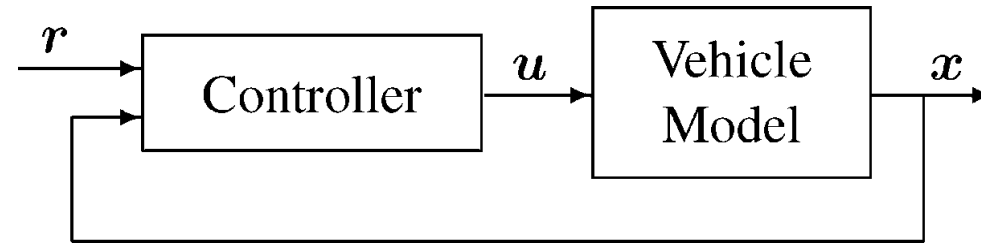
Basic Version of RRT (Lecture 4)

- **Sample:** Gives a sample in the free state space.
- **Nearest:** Provides the vertex in the tree that is closest to the sampled state.
- **Steer:** In general this is a so called two-point boundary value problem (TPBV). Construct a path from the nearest vertex towards the sampled state, often with a maximum path length (alternative strategies exist).
- **ObstacleFree:** Checks whether the path from the closest vertex in the graph to the new state is collision free.

Closed Loop Rapidly-Exploring Random Tree (CL-RRT)

Closed Loop Rapidly-Exploring Random Tree (CL-RRT)

CL-RRT samples an input to the stable closed-loop system consisting of the vehicle and the controller.



- CL-RRT works for vehicles with unstable dynamics, such as cars and helicopters, by using a stabilizing controller.
- A single input to the closed-loop system can create a long trajectory (on the order of several seconds) while the controller provides a high-rate stabilizing feedback

CL-RRT: Sampling strategies

Given a reference position and heading (x_0, y_0, θ_0) a sample point

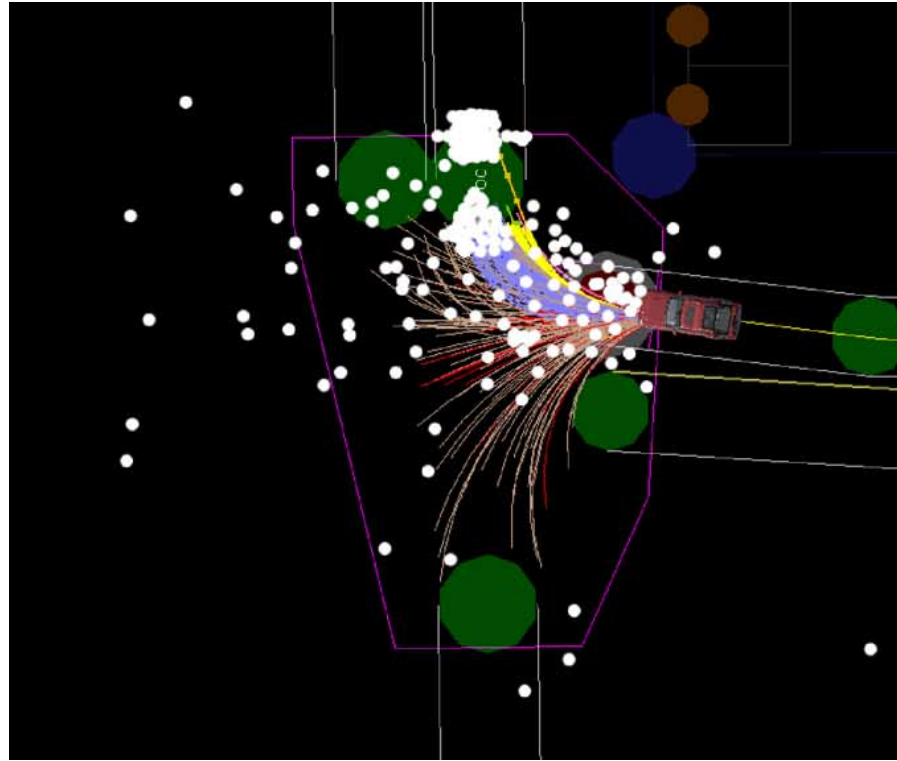
$$\begin{pmatrix} s_x \\ s_y \end{pmatrix} \text{ is generated by: } \begin{bmatrix} s_x \\ s_y \end{bmatrix} = \begin{bmatrix} x_0 \\ y_0 \end{bmatrix} + r \begin{bmatrix} \cos(\theta) \\ \sin(\theta) \end{bmatrix} \text{ with } \begin{cases} r = \sigma_r |n_r| + r_0 \\ \theta = \sigma_\theta n_\theta + \theta_0 \end{cases}$$

where n_r and n_θ are random variables with standard Gaussian distribution,

σ_r is the standard deviation in the radial direction σ_θ is the standard

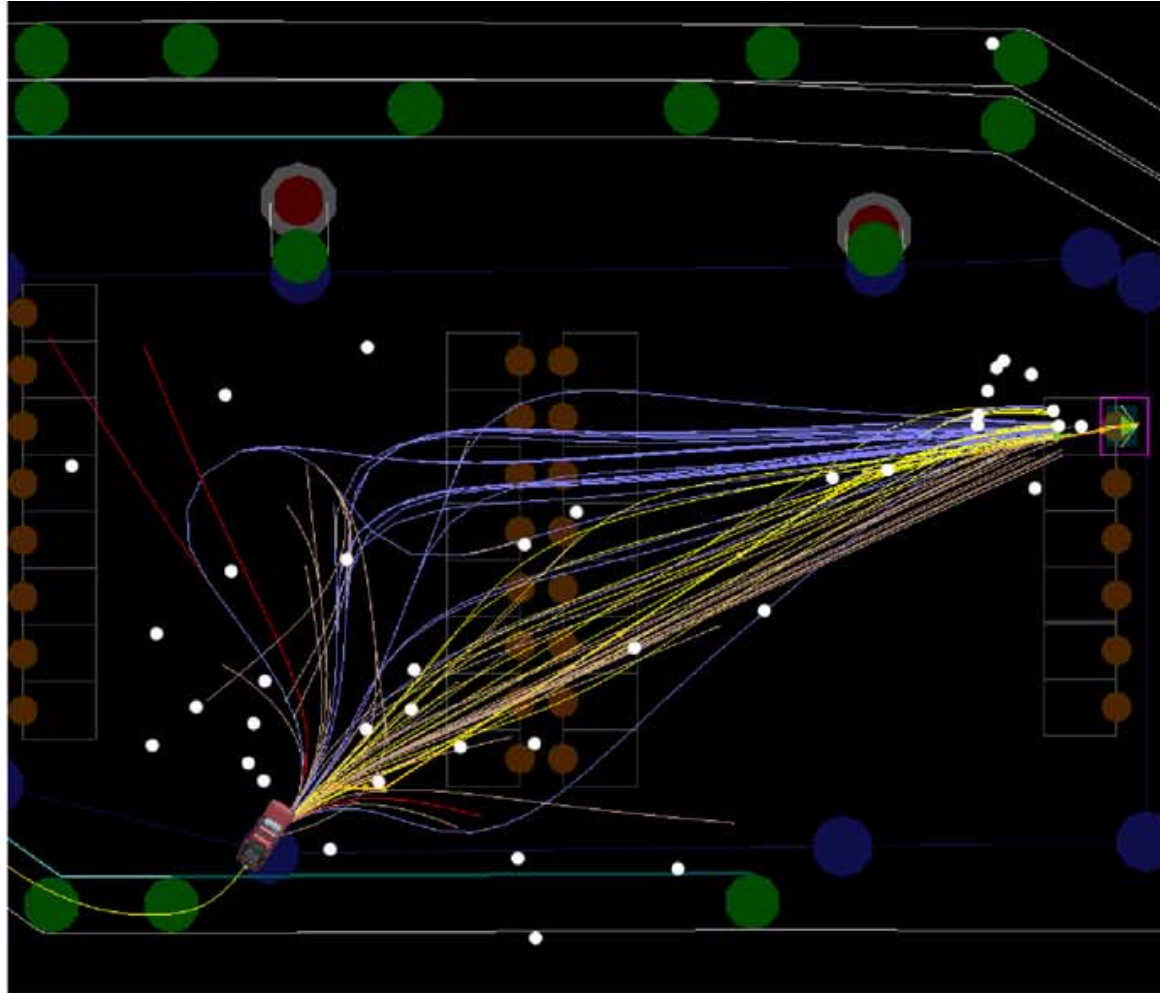
deviation in the circumferential directions, and r_0 is an offset with respect to

Example: Sampling at a right hand turn at an intersection⁷¹



A wide and relatively short Gaussian distribution is used that covers the open space inside the intersection boundary. The value of σ_r is set to the distance to the goal and σ_θ is set at 0.4π .

Example: Sampling at a parking lot

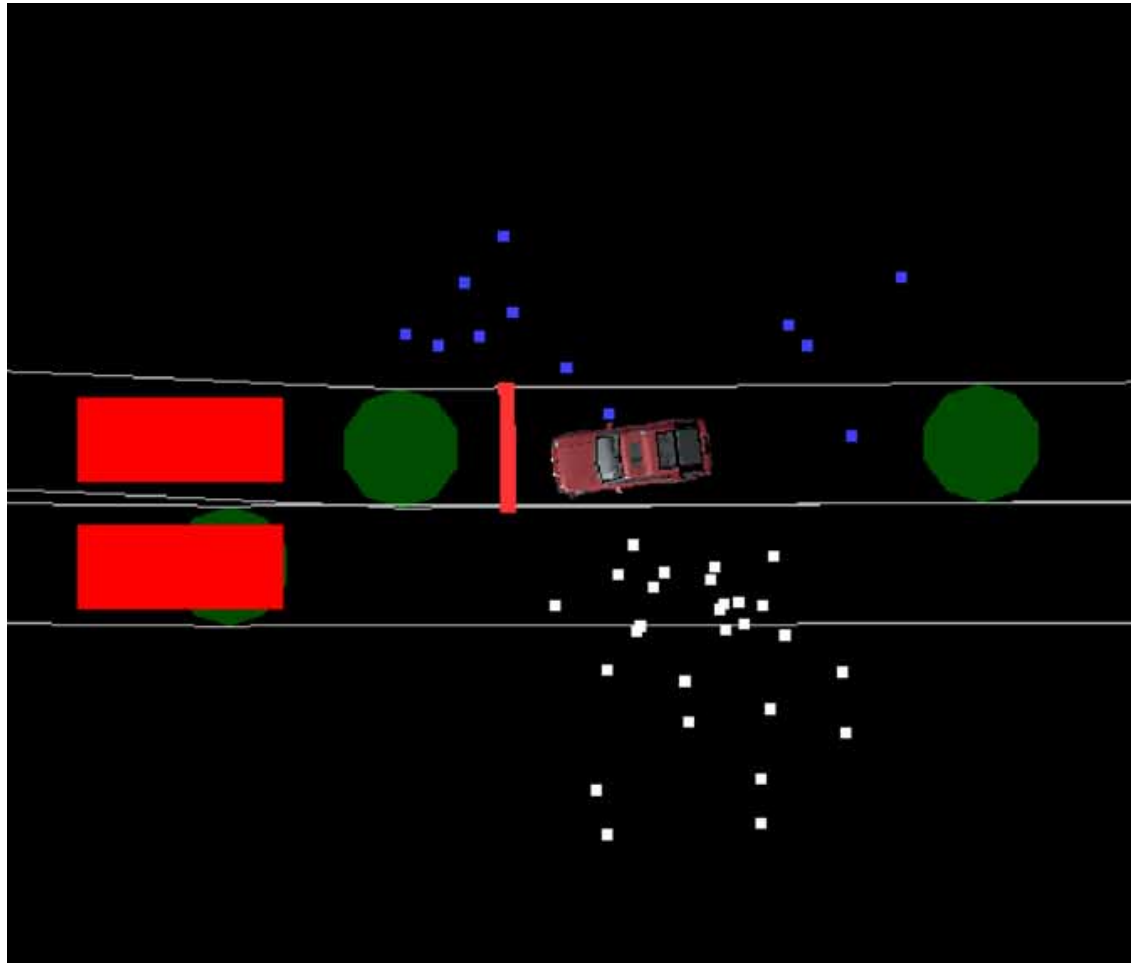


Sampling is taken both around the vehicle and around the parking spot.

Around the vehicle a wide and long distribution is used

Example: Sampling strategy for an U-turn

73

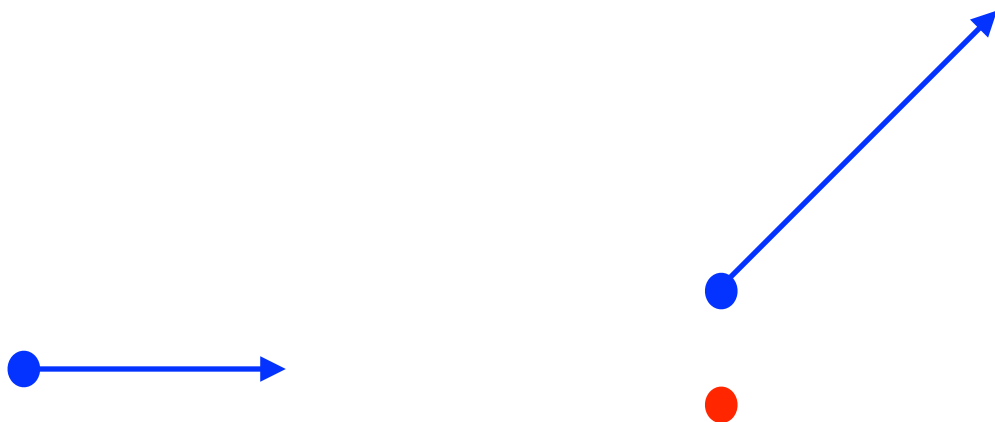


The vehicle is facing the road blockage (red).

Blue and white dots are reverse and forward manoeuvres, respectively.

CL-RRT: Heuristic

Which node (blue) is “closest” to the sample state (red)?

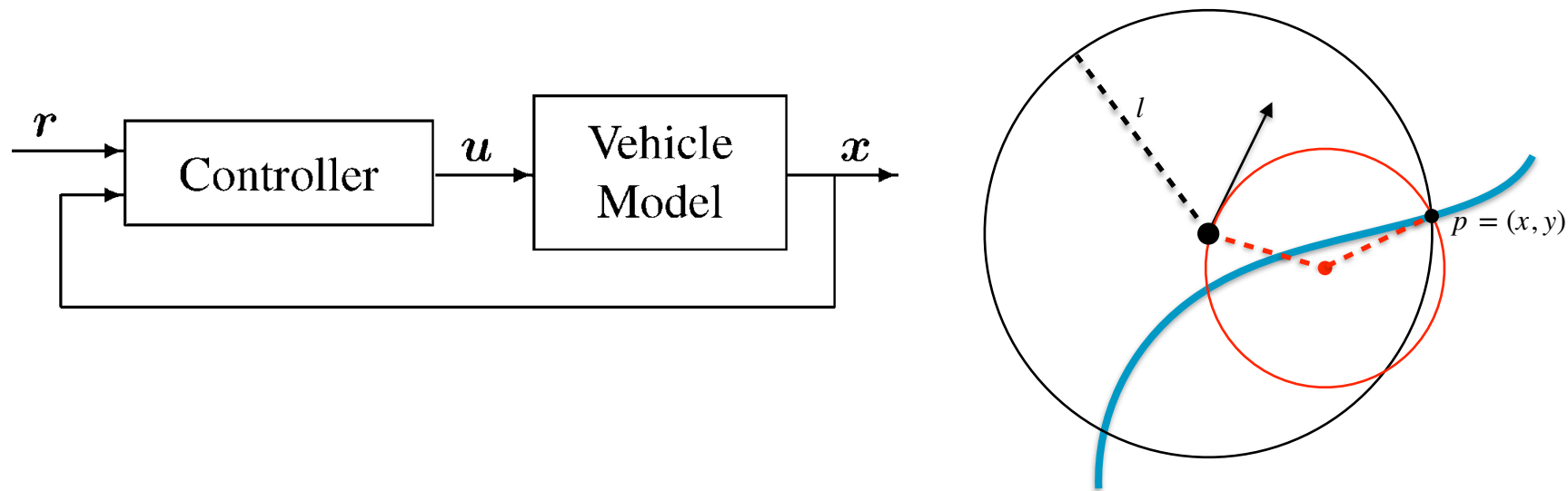


In RRT the distance was used, and the answer in this case the nearest node would be the one to the right.

In CL-RRT the Dubins distance is used, defined as the shortest path a with a minimal turning radius R_{min} . The nearest node would be the one to the left if R_{min} is sufficiently large. This is called the exploration heuristic.

CL-RRT: The controller

A pure-pursuit controller is used in the CL-RRT

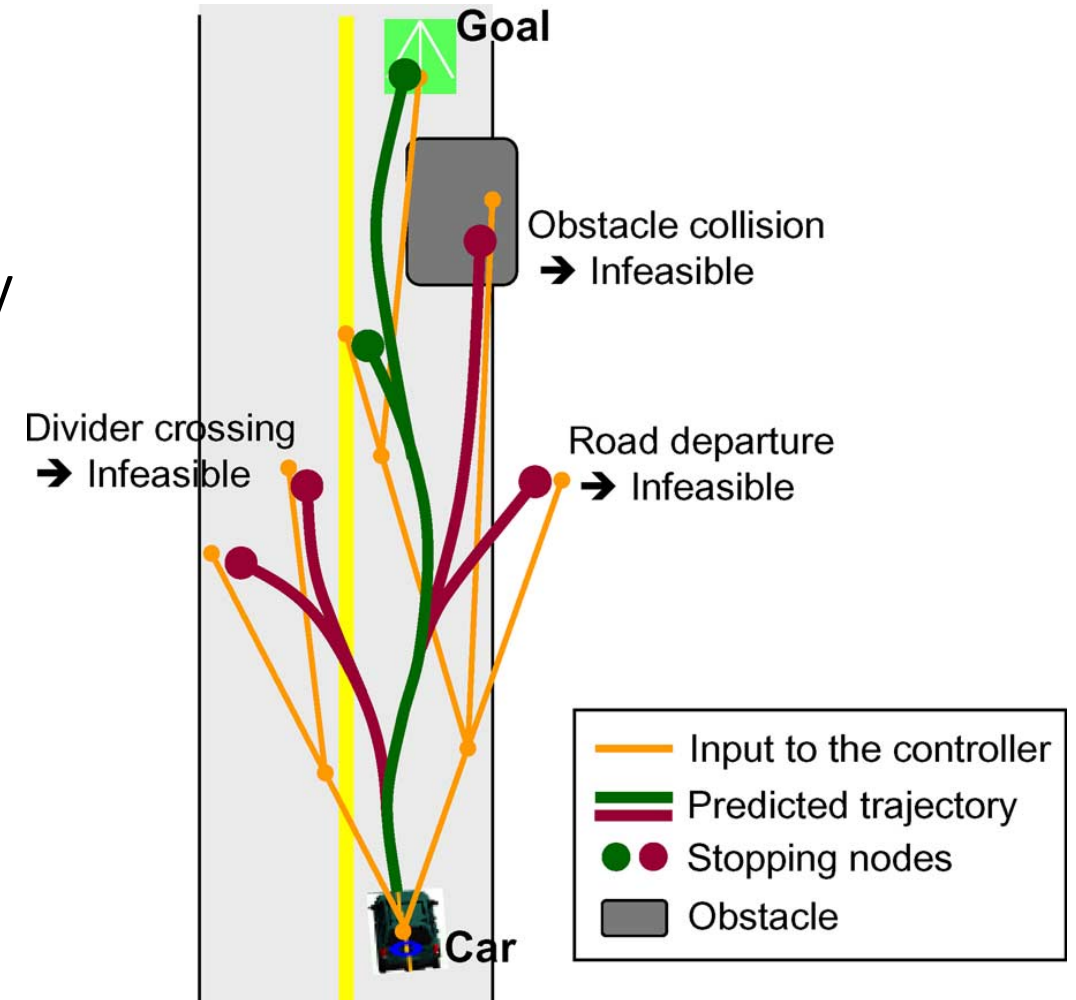


The next step is to describe how to construct reference paths for this controller.

CL-RRT: Expand the tree

Procedure to expand the (orange) tree

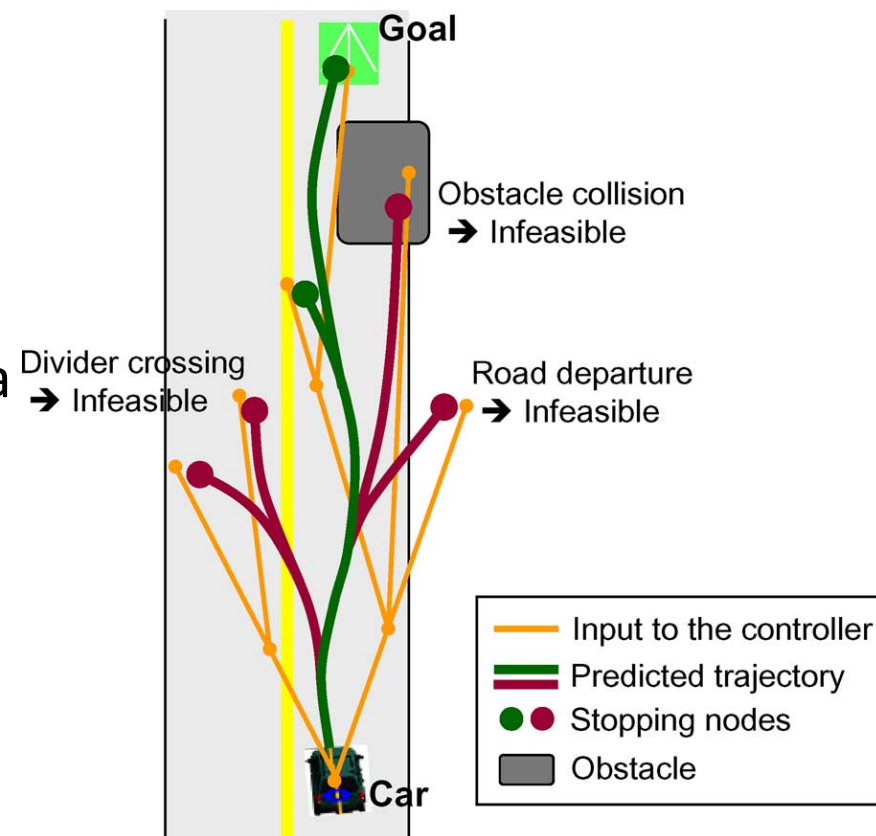
- Generate a sample position s .
- For each node q in the tree, in the order sorted by the Dubins distance $L_\rho(s)$, use the line segment between node q and sample s to extend the orange tree.
- Use the new reference path in the orange tree as input to the controller to generate a trajectory $\mathbf{x}(t)$, $t \in [t_1, t_2]$ until it stops (red and green curves).



CL-RRT: Expand the tree

- If $\mathbf{x}(t) \in \mathcal{X}_{free}(t)$ for all $t \in [t_1, t_2]$, then add the sample s to the tree and also some intermediate nodes.
- Else if the intermediate nodes are feasible, add them to the tree.

If no nodes were added above, then repeat the process with a new node q .

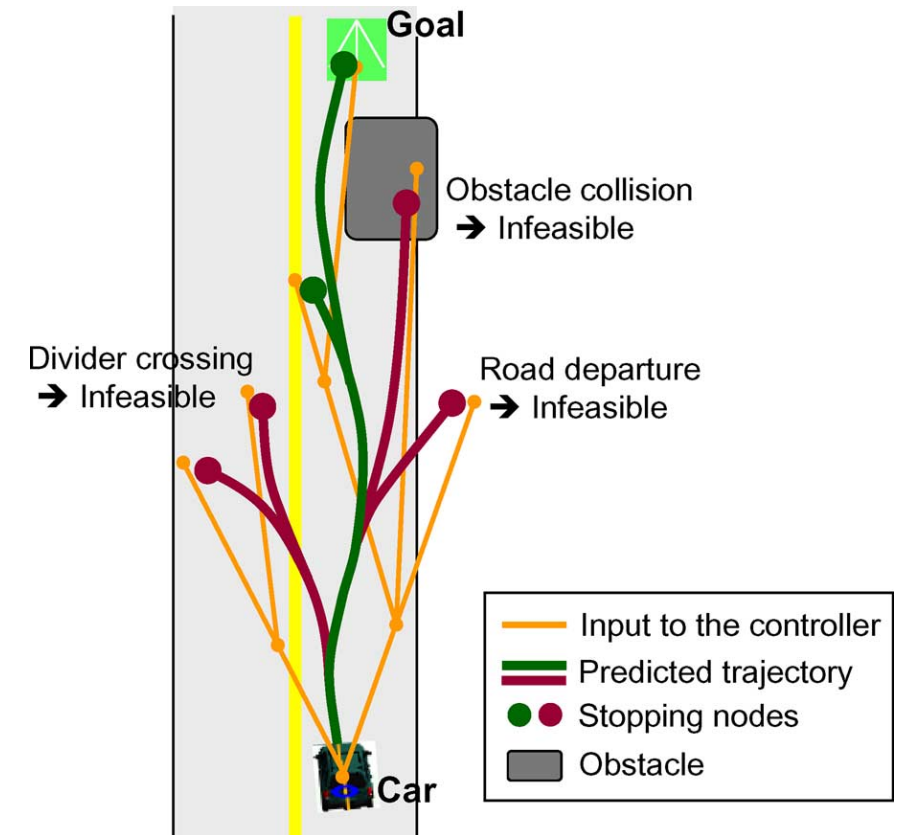


CL-RRT

The tree is expanded until the goal has been reached. After that the nodes are mainly sorted by ascending order of total cost to reach the sample:

$$C_{total} = C_{cum}(q) + L_{\rho}(s)/v$$

where $C_{cum}(q)$ is the cumulative cost from the root of the tree to a node q , $L_{\rho}(s)$ is the Dubins distance, and v is the sampled speed. The objective is to make the new trajectories approach the shortest path. This is called the optimisation heuristic.



Closed-Loop RRT for UAV Navigation in Dynamic Environments

Dan Levine, Brandon Luders, Jonathan How
Narrated by Karl Kulling

Aerospace Controls Laboratory, MIT
February 17, 2009



www.liu.se